Dynamic nearest-neighbor searching for general-metric homothets in three dimensions

Pankaj K. Agarwal, Matthew J. Katz, Micha Sharir

arxiv:2603.26585; CC BY 4.0

Editorial scope. The data structure combines a source-specific decomposition of distance-function k-levels, vertical shallow cuttings, fast-query dynamization and a shallow-partition linear-space construction to derive the storage/query/update tradeoff. Full native author proof retained under own CC BY4.0 license. Conference explicitly omitted proofs; this delivery uses the actual complete full-version source.

Allowed standard prerequisites (editorial).

- Standard semi-algebraic range searching, dynamization and parametric optimization techniques. Original author statements and citations retained; all source-specific k-level decomposition, partition construction and modified simulation-cost analysis present in full author source.

1 Introduction

Problem statement. Let K be a compact, centrally-symmetric, strictly-convex region in $\mathbb{R}^3$, which is a semi-algebraic set of constant complexity.[1] K is the unit ball of a norm $\|\cdot\|_K$, and we denote its metric as dist_K. A homothet (or homothetic copy) of K is

[1] Roughly speaking, a semi-algebraic set in $\mathbb{R}^d$ is the set of points in $\mathbb{R}^d$ that satisfy a Boolean formula over a set of polynomial inequalities; the complexity of a semi-algebraic set is the number of polynomials defining the set and their maximum degree. See [13] for formal definitions of a semi-algebraic set and its complexity.

a scaled and translated copy of K, expressed as $K(c,\rho) := \rho K + c$, for $c \in \mathbb{R}^3$ and $\rho \geq 0$; c is the center of $K(c,\rho)$ and ρ is its size. We represent $K(c,\rho)$ as the point $(c,\rho) \in \mathbb{R}^3 \times \mathbb{R}_{\geq 0}$. Let $\mathbb{K}$ be the set of all homothets of K, represented as the halfspace $x_4 \geq 0$ of $\mathbb{R}^4$. The K-distance $\text{dist}_K(u,v) = \|u-v\|_K$ between points $u, v \in \mathbb{R}^3$ is the minimum value of ρ for which $v \in K(u,\rho)$ (or, equivalently, $u \in K(v,\rho)$). Since K is strictly convex, the triangle inequality $\text{dist}_K(u,v) \leq \text{dist}_K(u,w) + \text{dist}_K(w,v)$ is sharp unless w lies on the segment uv. The K-distance of a point $x \in \mathbb{R}^3$ from a homothet $K(c,\rho)$ is defined as $f_{c,\rho}(x) = \text{dist}_K(c,x) - \rho$, and the K distance of another homothet $K(c',\rho')$ from $K(c,\rho)$ is $f_{c,\rho}(c') - \rho' = \text{dist}_K(c,c') - \rho - \rho'$. Since K is a constant-complexity semi-algebraic set, $f_{c,\rho}$ is a semi-algebraic function of constant complexity. The homothets $K(c,\rho)$ and $K(c',\rho')$ intersect if the distance between them is non-positive, i.e., $\text{dist}_K(c,c') \leq \rho + \rho'$, or equivalently, the point (c',ρ') lies above the graph of $f_{c,\rho}$ in $\mathbb{R}^4$, or vice-versa. Let $\mathcal{K} = \{K_1, \ldots, K_n\} \subset \mathbb{K}$ be a set of n homothets of K, where $K_i = K(c_i, \rho_i)$. In this paper we study two classes of proximity problems related to $\mathcal{K}$.

Proximity queries. We wish to preprocess $\mathcal{K}$ into a data structure so that various proximity queries for a query homothet K_0, such as intersection-detection (determine whether K_0 intersects any homothet of $\mathcal{K}$; if the answer is yes, return one such intersecting homothet of $\mathcal{K}$), intersection reporting (report all homothets of $\mathcal{K}$ that intersect K_0), and nearest-neighbor queries (report the homothet of $\mathcal{K}$ at the minimum K-distance from K_0), can be performed efficiently. In addition, the data structure should support insertions and deletions of homothets of K.

Searching in the intersection graph and related problems. Let $\mathcal{G}(\mathcal{K})$ denote the intersection graph of $\mathcal{K}$, whose vertices are the homothets of $\mathcal{K}$ and (K_i, K_j) is an edge if $K_i \cap K_j \neq \emptyset$. Similarly, for a threshold parameter r_0, the r_0-proximity graph $\mathcal{G}_{r_0}(\mathcal{K})$ of $\mathcal{K}$ consists of those edges (K_i, K_j) for which $\text{dist}_K(K_i, K_j) \leq r_0$. We study efficient implementation of graph-search problems on $\mathcal{G}(\mathcal{K})$, like BFS or DFS. We also study the reverse shortest path (RSP) problem on $\mathcal{K}$: given two elements $K_i, K_j \in \mathcal{K}$ and a parameter $1 \leq k < n$, compute the smallest value r^* for which $\mathcal{G}_{r^*}(\mathcal{K})$ contains a path from

K_i to K_j with at most k edges.

Related work. There is extensive work on nearest-neighbor (NN) searching in many different fields including computational geometry, database systems, and machine learning. We refer the readers to various survey papers [7, 12, 23] for a general overview of known results on this topic. Here we briefly mention a few results that are most closely related to the problems studied in this paper. Agarwal and Matoušek [11] presented a dynamic NN data structure for a point set in $\mathbb{R}^d$, under the Euclidean metric, that answers a query in $O^*(n/s^\phi)$ time, where $\phi = 1 - \frac{1}{\lceil d/2 \rceil}$ and $s \in [n, n^{\lceil d/2 \rceil}]$ is a storage parameter, using $O^*(s)$ space and preprocessing; the (amortized) update time for insertions/deletions of points is $O^*(s/n)$. The bounds were slightly improved – n^ε factors were replaced by $\log^{O(1)} n$ factors – in [18, 28]. Kaplan et al. [28] presented a dynamic NN data structure for a set S of points in $\mathbb{R}^2$ under fairly general distance functions. Its performance depends on the complexity of the Voronoi diagram of S under that distance function. For the Minkowski metric induced by a centrally-symmetric convex region, which is a semi-algebraic set of constant complexity, their data structure can answer an NN query in $O(\log^2 n)$ time, and handle insertion and deletion of a point in $O(\beta(n) \log^5 n)$ and $O(\beta(n) \log^9 n)$ amortized expected time, respectively, where $\beta(n)$, a variant of inverse Ackermann's function, is an extremely slowly growing function; see also [33] for slightly improved bounds. These data structures, however, do not extend to $\mathbb{R}^3$. The problem of NN searching under general distance functions in $\mathbb{R}^3$ is much less understood. By reducing NN queries to ball-intersection queries [10] and using semi-algebraic range-searching data structures [2, 3, 35], an NN query amid the set $\mathcal{K}$ of n homothets in $\mathbb{R}^3$, as above, can be answered in $O^*(1)$ time using $O^*(n^4)$ space and preprocessing, or in $O^*(n^{3/4})$ time using $O(n)$ space and preprocessing. Using recent standard techniques (see, e.g., the Appendix in [2]), one can obtain a space/query-time trade-off, as well as handle insertions/deletions of objects. Using a recent result by Agarwal et al. [5] on the vertical decomposition of the lower envelope of trivariate functions, an NN query amid $\mathcal{K}$ can be answered in $O(\log^2 n)$ time using only $O^*(n^3)$ space, but this data structure does not support efficient deletions (insertions can

be handled in a standard manner, using the dynamization technique of Bentley and Saxe [15]). Furthermore, it was not known whether an NN query amid $\mathcal{K}$ can be answered in $O^*(n^{2/3})$ time using $O^*(n)$ space. Motivated by numerous applications, there has been work on developing fast algorithms for intersection and proximity graphs of n geometric objects. Although the intersection graph may have $\Theta(n^2)$ edges, the goal in this line of work is to develop algorithms, by exploiting the underlying geometry, that run in $O^*(n)$ time (or at least in subquadratic time). Cabello and Jejčič [17], and subsequently Chan and Skrepetos [24], presented $O(n \log n)$ implementations of BFS in unit-disk intersection graphs, which was recently extended to general disk-intersection graphs by de Berg and Cabello [16]; see also [27, 28, 31]. The algorithm in [16] can also perform DFS and Dijkstra's algorithm in $O(n \log n)$ time. Katz et al. [30] showed that BFS in ball-intersection graphs (for congruent or arbitrary Euclidean balls) can be implemented in $O^*(n^{2\phi/(\phi+1)})$ time, where $\phi = \lfloor d/2 \rfloor + 1$, so in $O^*(n^{4/3})$ time for $d = 3$. The problem of devising efficient implementations for BFS/DFS in more general graphs in the plane also has received some attention, see, e.g., [8, 9, 29]. The reverse shortest path (RSP) problem for unit disks was studied by Wang and Zhao [38], who gave an $O^*(n^{5/4})$-time solution. Kaplan et al. [27] improved the runtime to $O^*(n^{6/5})$ using the shrink-and-bifurcate technique (see [14]), and it was recently improved to $O^*(n^{9/8})$ by Chan and Huang [20]. The best-known RSP algorithm for arbitrary disks runs in $O^*(n^{6/5})$ expected time. Katz et al. [30] presented RSP algorithms for balls in $\mathbb{R}^3$ — with $O^*(n^{29/21})$ time for congruent balls and $O^*(n^{56/39})$ for arbitrary balls. The RSP problem also has been studied for other geometric objects in $\mathbb{R}^2$ [9, 20, 30].

Our results. Our main result is a dynamic data structure for answering intersection and NN queries with a homothet of $\mathbb{K}$ amid the set $\mathcal{K}$ of homothets, as stated in the following theorem:

Theorem 1.1. Let K be a compact, centrally-symmetric strictly convex semi-algebraic set in $\mathbb{R}^3$ of constant complexity. Let $\mathcal{K}$ be a set of n homothetic copies of K. For a storage parameter $s \in [n, n^3]$, $\mathcal{K}$ can be maintained in a dynamic data structure with $O^*(s)$ storage, that can answer an intersection-detection or NN-query (under the K-distance) with a homothet K_0 of K in $O^*(n/s^{1/3})$ time and

that supports insertions/deletions in $O^*(s/n)$ amortized expected time. It can report all k objects of $\mathcal{K}$ intersecting K_0 in additional $O(k)$ time.

Plugging Theorem 1.1 into Eppstein's technique [25], or its enhancement by Chan [19], for maintaining bichromatic closest pairs, using $s = n^{3/2}$, we obtain:

Corollary 1.2. Let K be a compact, centrally-symmetric strictly-convex semi-algebraic set in $\mathbb{R}^3$ of constant complexity. Let $\mathcal{K}, \mathcal{K}'$ be two sets of homothets of K of combined size n. The K-closest pair between $\mathcal{K}$ and $\mathcal{K}'$, under insertions/deletions of homothets (in $\mathcal{K}$ and $\mathcal{K}'$), can be maintained in $O^*(n^{1/2})$ amortized expected time per update.

Theorem 1.1 is obtained by presenting two data structures. The first one (see Section 4.1) uses $O^*(n^3)$ space, answers a query in $O^*(1)$ time, and handles an insertion/deletion in $O^*(n^2)$ amortized expected time. (The data structure in [5] also obtains a similar space/query-time bound but it cannot handle deletions efficiently.) This data structure is based on constructing vertical shallow cuttings, a notion originally introduced in [18, 21], of the graphs of the distance functions of $\mathcal{K}$. Roughly speaking, for a parameter $t \geq 1$, a vertical (n/t)-shallow cutting of $\mathcal{K}$ is a collection of pairwise openly disjoint semi-unbounded pseudo-prisms that covers the region lying below the k-level of the arrangement of $\mathcal{K}$, where each prism, consisting of all points that lie vertically below a constant-complexity semi-algebraic 3-dimensional region, intersects the graphs of only $O(n/t)$ functions of $\mathcal{K}$. The random-sampling based technique used in [18, 28] to construct a vertical shallow cutting does not immediately extend to our setting because one needs to decompose the q-level, for some parameter $q > 0$, in the 4D arrangement of the distance functions of $\mathcal{K}$ into $O^*(n^3(q+1))$ constant-complexity cells. Although a recent result [5] constructs such a decomposition of the 0-level (with $O^*(n^3)$ cells), it does not extend to larger values of q. This problem remains elusive for arrangements of arbitrary trivariate semi-algebraic functions of constant complexity, but we develop a desired decomposition technique for our setting (in Section 2) by exploiting the structural geometric properties of the distance functions of $\mathcal{K}$. This result is one of the main technical contributions

of the paper. Using our result on the decomposition of the q-level, we construct a vertical (n/t)-shallow cutting of $\mathcal{K}$ of size $O^*(t^3)$, for any t (Section 3). We build a tree data structure using vertical shallow cuttings and adapt the technique in [11] to handle efficient deletions of homothets from $\mathcal{K}$; as noted, insertions are handled using the standard Bentley-Saxe technique [15]. The second data structure is a linear-size partition tree, with $O^*(n^{2/3})$ query time, constructed on the point set $\mathcal{K}^* = \{(c_i, \rho_i) \mid 1 \leq i \leq n\} \subset \mathbb{R}^4$, that answers intersection-detection queries on $\mathcal{K}$ with a homothet of $\mathbb{K}$ as a query. The main technical challenge we face here is the construction of a so-called test set of $\mathcal{K}$ (see [37] and below) of size $r^{O(1)}$, which represents well the distance functions of all (n/r)-shallow homothets in $\mathbb{K}$, i.e., the homothets that intersect at most n/r elements of $\mathcal{K}$. By adapting the technique in [37] and again exploiting the structural geometric properties of the distance functions, we show (in Section 4.2) that a test set, of size $O^*(r^4)$, with the desired properties can be constructed efficiently. Using Theorem 1.1 and Corollary 1.2, we obtain our second set of results related to searching in the intersection and proximity graphs of $\mathcal{K}$.

Theorem 1.3. (a) Given a set $\mathcal{K}$ of n homothets of a constant-complexity, strictly convex, centrally-symmetric semi-algebraic set K, BFS or DFS in the intersection graph of $\mathcal{K}$ can be performed in $O^*(n^{3/2})$ expected time. (b) The single-source shortest-path tree as well as the minimum spanning forest of the r_0-proximity graph of a set of n points, for any threshold parameter r_0, weighted by the pairwise K-distances, can be computed in $O^*(n^{3/2})$ time.

Combining Theorem 1.3(a) with the shrink-and-bifurcate technique [14, 27] yields:

Theorem 1.4. Given a set $\mathcal{K}$ of n homothets of a constant-complexity, strictly convex, centrally-symmetric semi-algebraic set K, two designated elements K, $K' \in \mathcal{K}$, and an integer $k < n$, the RSP problem on $\mathcal{K}$, of finding the smallest r^* for which $G_{r^*}(\mathcal{K})$ contains a path between K and K' of length at most k, can be solved in $O^*(n^{62/39})$ expected time.

2 Decomposition of the k-Level of an Arrangement of Distance Functions

Preliminaries. Let $\mathcal{K} = \{K_1, \ldots, K_n\}$, where each K_i is a homothetic copy of K represented by a point $K_i^* = (c_i, \rho_i)$ in $\mathbb{R}^4$. For $1 \leq i \leq n$, set $f_i = f_{c_i,\rho_i}$, and let $\mathcal{F} := \mathcal{F}(\mathcal{K}) = \{f_i \mid 1 \leq i \leq n\}$. (As noted, distances from points x that lie inside some copy K_i, still measured by $f_i(x)$, are nonpositive.) We will not distinguish between a function of $\mathcal{F}$ and its graph. For a point $x \in \mathbb{R}^3$, K_i is the nearest neighbor of x in $\mathcal{K}$ (under the K-distance dist_K), i.e., $i = \arg\min_{1 \leq j \leq n} f_j(x)$, if f_i appears on the lower envelope of $\mathcal{F}$ at x. The level of a point $p \in \mathbb{R}^4$ in $\mathcal{A}(\mathcal{F})$ is the number of functions in $\mathcal{F}$ whose graphs lie below p. For a parameter $k < n$, the k-level of $\mathcal{A}(\mathcal{F})$, denoted $\mathcal{A}_k(\mathcal{F})$, is the (closure of the) locus of all points on $\bigcup \mathcal{F}$ whose level is k. We define the $(\leq k)$-level of $\mathcal{A}(\mathcal{F})$, denoted $\mathcal{A}_{\leq k}(\mathcal{F})$, to be the (closure of the) set of all points in $\mathbb{R}^4$ of level at most k, i.e., the set of points that lie on or below $\mathcal{A}_k(\mathcal{F})$ (not necessarily on $\cup\mathcal{F}$). The projection of $\mathcal{A}_k(\mathcal{F})$ onto $\mathbb{R}^3$, denoted by $\mathcal{M}_k = \mathcal{M}_k(\mathcal{K})$, is a subdivision of $\mathbb{R}^3$. If a cell $\tau^\downarrow$ of $\mathcal{M}_k$ is the projection of a cell τ of $\mathcal{A}_k(\mathcal{F})$ that lies on the graph of f_i, then K_i is the k-th nearest neighbor in $\mathcal{K}$ (under the K-distance function) of all points in $\tau^\downarrow$. Note that $A_0(\mathcal{F})$ is the lower envelope of $\mathcal{F}$, and $\mathcal{M}_0(\mathcal{K})$ is the Voronoi diagram of $\mathcal{K}$ (under the K-distance).

The decomposition. The main technical result of this section is that $\mathcal{A}_k(\mathcal{F})$ can be decomposed into $O^*(n^3(k+1))$ elementary cells, namely, constant-complexity semi-algebraic regions, each homeomorphic to a ball; see below for a more precise definition.

Lemma 2.1. *K_i is contained in K_j if and only if $\mathrm{dist}_K(c_j, c_i) \leq \rho_j - \rho_i$.*

Proof. Suppose that $\mathrm{dist}_K(c_j, c_i) > \rho_j - \rho_i$. Let x be the intersection point of ∂K_i with the ray emanating from c_i in direction $c_i - c_j$, i.e., lying on the line passing through c_i and c_j but not containing c_j. Because of the collinearity of c_i, c_j, and x,

$$\mathrm{dist}_K(c_j, x) = \mathrm{dist}_K(c_j, c_i) + \mathrm{dist}_K(c_i, x) = \mathrm{dist}_K(c_j, c_i) + \rho_i > \rho_j.$$

Hence, $x \notin K_j$ and thus $K_i \not\subseteq K_j$. Conversely, assume that $\mathrm{dist}_K(c_j, c_i) \leq \rho_j - \rho_i$. Let $x \in K_i$ be an arbitrary point. That is,

$\text{dist}_K(x, c_i) \leq \rho_i$. By the triangle inequality we have

$$\begin{aligned}\text{dist}_K(x, c_j) - \rho_j &\leq \text{dist}_K(x, c_i) + \text{dist}_K(c_i, c_j) - \rho_j \\ &\leq \text{dist}_K(x, c_i) + (\rho_j - \rho_i) - \rho_j \\ &= \text{dist}_K(x, c_i) - \rho_i \leq 0,\end{aligned}$$

i.e., $x \in K(c_j, \rho_j)$. Since this holds for any point $x \in K_i$, we conclude that $K(c_i, \rho_i) \subseteq K(c_j, \rho_j)$. □

Lemma 2.2. Let K_i and K_j be a pair of non-nested homothets in $\mathcal{K}$ (i.e., neither of them lies inside the other). Then $f_i(c_i) < f_j(c_i)$.

Proof. Assume to the contrary that $f_j(c_i) \leq f_i(c_i)$. That is,

$$\text{dist}_K(c_j, c_i) - \rho_j \leq \text{dist}_K(c_i, c_i) - \rho_i = -\rho_i, \qquad \text{or} \qquad \text{dist}_K(c_j, c_i) \leq \rho_j - \rho_i.$$

But then, by Lemma 2.1, $K_i \subseteq K_j$, contradicting the assumption that they are non-nested. □

For $1 \leq i \neq j \leq n$, let $B_{ij} = \{x \in \mathbb{R}^3 \mid f_i(x) = f_j(x)\}$ be the bisector of K_i and K_j under the K-distance. Note that if $K_i \subset K_j$ then, as is easily verified, $f_j(x) < f_i(x)$ for all $x \in \mathbb{R}^3$ and B_{ij} is undefined. The following lemma gives a useful property of bisectors.

Lemma 2.3. Fix a homothet $K_i \in \mathcal{K}$. Let λ be a ray in $\mathbb{R}^3$ emanating from c_i. For any $1 \leq j \neq i \leq n$ such that $K_i \not\subset K_j$, λ intersects B_{ij} in at most one point, say, ξ. Furthermore, $f_i(x) \leq f_j(x)$ for all $x \in c_i\xi$ (where $c_i\xi$ denotes the segment with endpoints c_i and ξ) and $f_j(x) > f_i(x)$ for all $x \in \lambda \setminus c_i\xi$.

Proof. Suppose to the contrary that λ intersects B_{ij} at two points $x \neq y$, and assume, without loss of generality, that x is closer to c_i than y; see Figure 1. We then have

$$\begin{aligned}\text{dist}_K(x, c_i) - \rho_i &= \text{dist}_K(x, c_j) - \rho_j \\ \text{dist}_K(y, c_i) - \rho_i &= \text{dist}_K(y, c_j) - \rho_j,\end{aligned}$$

and thus

$$\text{dist}_K(y, c_i) - \text{dist}_K(x, c_i) = \text{dist}_K(y, c_j) - \text{dist}_K(x, c_j).$$

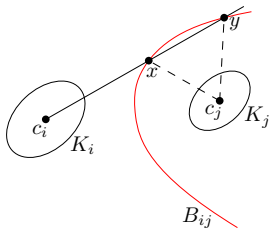


Figure 1: Illustrating (an impossible scenario in) the proof of Lemma 2.3.

But the left-hand side is $\mathrm{dist}_K(x, y)$ (because of the collinearity of c_i, x, and y), so we get

$$\mathrm{dist}_K(x, y) = \mathrm{dist}_K(y, c_j) - \mathrm{dist}_K(x, c_j).$$

This however is impossible in general, because of the strict convexity of K, unless x, y and c_j are also collinear, that is, when λ is the ray from c_i through c_j. Regardless of which ray λ is, since $f_i(c_i) < f_j(c_i)$ (by Lemma 2.2), $f_i(z) \leq f_j(z)$ for all $z \in c_i x$ and $f_j(z) < f_i(z)$ for all $z \in \lambda \setminus c_i x$. On the other hand, if λ does pass through c_i and c_j, then there is a unique point on the line $\bar{\lambda}$ supporting λ that is equidistant from K_i and K_j. Indeed, by Lemma 2.2, $f_i(c_i) < f_j(c_i)$ and $f_j(c_j) < f_i(c_j)$, so B_{ij} intersects the segment $c_i c_j$ in at least one point, say x. As we move along $\bar{\lambda}$ from x toward c_j, the value of f_j decreases but the value of f_i increases until we reach c_j, and then, beyond c_j, both functions increase at the same rate, so $f_j(y) < f_i(y)$ for all points y on $\bar{\lambda}$ after x and, arguing symmetrically, $f_i(y) < f_j(y)$ for all points on $\bar{\lambda}$ before x. This completes the proof of the lemma. □

The following lemma is an easy corollary of Lemma 2.3.

Lemma 2.4. Let τ be a three-dimensional cell of $\mathcal{A}_k(\mathcal{F})$ lying on the graph of a function $f_i \in \mathcal{F}$, and let $\tau^{\downarrow}$ denote the projection of τ onto $\mathbb{R}^3$. Then $\tau^{\downarrow}$ is monotone with respect to c_i, in the sense that any ray emanating from c_i intersects $\tau^{\downarrow}$ in a connected (possibly empty or unbounded) segment.

We are now ready to describe the decomposition of the cells of $\mathcal{A}_k(\mathcal{F})$ into elementary cells. (Our ultimate goal is to decompose the region below $\mathcal{A}_k(\mathcal{F})$ into elementary cells, but we start with this subtask.) For $1 \le i \le n$, let $\mathcal{A}_k^i = \mathcal{A}_k^i(\mathcal{F})$ denote the subset of cells of $\mathcal{A}_k(\mathcal{F})$ that lie on the graph of f_i, and let $\mathcal{M}_k^i$ denote their projections (which are the cells of $\mathcal{M}_k$ for which K_i is the k-th nearest neighbor). We describe the algorithm for decomposing the cells of $\mathcal{M}_k^i$ (or of $\mathcal{A}_k^i$). Lemma 2.3 implies that, for any $j \neq i$, B_{ij} can be viewed as the graph of a function $g_{ij} : \mathbb{S}^2 \to \mathbb{R}_{\ge 0}$ in spherical coordinates with c_i as the origin.[2] That is, for a given $u \in \mathbb{S}^2$, let $\lambda(u)$ be the ray emanating from c_i in direction u, and let $x_{ij}(u)$ be the intersection point of $\lambda(u)$ with the bisector B_{ij} if such an intersection point exists; otherwise $x_{ij}(u)$ is undefined. We set $g_{ij}(u) = f_j(x_{ij}(u))$ if $x_{ij}(u)$ is defined and $+\infty$ otherwise. Let $G^{(i)} = \{g_{ij} \mid 1 \le j \neq i \le n\}$. We define the level of a point $x \in \mathbb{R}^3$ in (the 3D arrangement, within $\mathbb{S}^2 \times \mathbb{R}$) $\mathcal{A}(G^{(i)})$ to be the number of functions in $G^{(i)}$ whose graphs intersect the (relatively open) segment $c_i x$. The following lemma will be useful in analyzing the complexity of the decomposition of $\mathcal{A}_k^i$.

Lemma 2.5. For a parameter $k < n$, let τ be a 3D cell of $\mathcal{A}_k^i$. Then $\tau^\downarrow$ is a cell of level at most k in $\mathcal{A}(G^{(i)})$.

Proof. It is obvious from the definition of $G^{(i)}$ that $\tau^\downarrow$ is a cell in $\mathcal{A}(G^{(i)})$. We now argue that its level in $\mathcal{A}(G^{(i)})$ is at most k. Choose a point $x \in \tau^\downarrow$. Let λ be the ray emanating from c_i and passing through x. Let g_{ij} be a function whose graph intersects the segment $c_i x$, say, at a point $x_0 \in B_{ij}$. (See Fig. 2.) By Lemma 2.3, $f_i(y) < f_j(y)$ for all points y on λ preceding x_0, and $f_j(y) < f_i(y)$ for all points on λ beyond x_0. In particular, $f_i(x) > f_j(x)$. Since τ is a cell of $\mathcal{A}(\mathcal{F})$, f_j does not intersect τ, so f_j lies below τ in $\mathbb{R}^4$ (in the x_4-, or rather ρ-direction). Furthermore, $\tau \in \mathcal{A}_k(\mathcal{F})$, so there are exactly k functions of $\mathcal{F}$ whose graphs lie below τ, implying that the level of $\tau^\downarrow$ in $\mathcal{A}(G^{(i)})$ is at most k, as claimed. □

We decompose the cells τ of $\mathcal{A}_k^{(i)}(\mathcal{F})$ by constructing "vertical decompositions" of the corresponding 3D projections $\tau^\downarrow$ in the

[2] To make everything semi-algebraic, we should reparameterize the spherical coordinates (θ, φ) using some (standard) algebraic representation. Nevertheless, for convenience, we stick to the spherical coordinate notation.

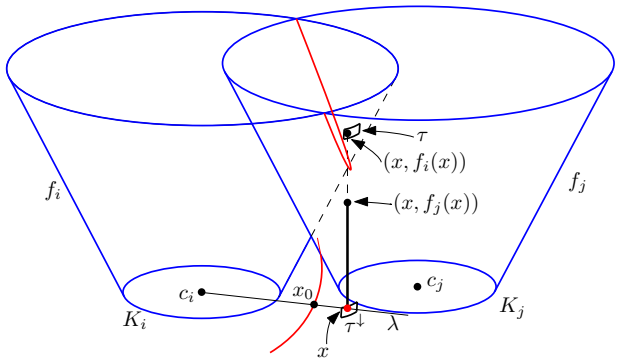


Figure 2: A bisector crossing λ. (The figure is drawn in the 3D c-space.)

spherical coordinate system around c_i. As for standard vertical decompositions in 3D, we proceed in two stages. The first stage erects a wall from each edge of τ as follows. Fix an edge e of $\tau^\downarrow$. For a point $x \in e$, let $\lambda(x)$ be the ray emanating from c_i and passing through x, and let $h(x) = \lambda(x) \cap \tau^\downarrow$. By Lemma 2.3, $h(x)$ is a (connected) segment that touches e at one of its endpoints — it is either the top endpoint, for all points on e, or the bottom endpoint (with respect to distances from c_i). We erect the wall $\hat{e} = \bigcup_{x \in e} h(x)$ in $\tau^\downarrow$; parts of the wall may extend to infinity when $\tau^\downarrow$ is unbounded and e is a bottom edge. We repeat this step for all edges of $\tau^\downarrow$. These walls decompose $\tau^\downarrow$ into frustums (truncated cones), each of which, denoted by Δ, has a unique pair of front and back faces; the other faces of Δ lie on the created walls. (See Fig. 3.) The projections of the front and back faces of Δ on $\mathbb{S}^2$ (with c_i as the origin) are identical, and we denote this identical projection by $\Delta^\downarrow$. The complexity of $\Delta^\downarrow$ may be arbitrarily large (a typical situation in the first stage of vertical decompositions in general; see, e.g., [22]). The subcell Δ itself is of the form

$$\Delta = \{(u, r) \mid u \in \Delta^\downarrow,\, r \in [g^-(u), g^+(u)]\},$$

where g^-, g^+ are the functions of $G^{(i)}$ whose graphs contain the front and back faces of Δ, respectively. The second stage decomposes Δ into constant-complexity frustums, which we refer to as pseudo-cones. We decompose $\Delta^\downarrow$ into spherical pseudo-trapezoids, by drawing portions of meridians within $\Delta^\downarrow$ from each

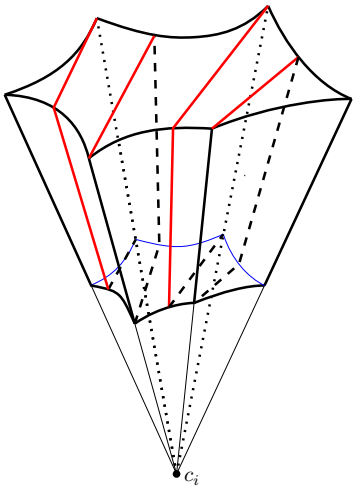


Figure 3: Vertical decomposition of a cell in the spherical coordinate system.

vertex and meridian-tangency point. We then lift each pseudo-trapezoid $\sigma^{\downarrow}$ to form the pseudo-cone $\sigma = \{(u,r) \mid u \in \sigma^{\downarrow}, r \in [g^-(u), g^+(u)]\}$ in $\mathbb{R}^3$. Repeating this step for all frustums created in the first stage, we obtain a decomposition of $\tau^{\downarrow}$ into pseudo-cones, each being a semi-algebraic set of constant complexity and bounded by up to six facets (compare with standard vertical decompositions in 3D [22]). We refer to these pseudo-cones as elementary cells. We note that each pseudo-cone ϕ is defined by a subset D_ϕ of at most seven functions of $\mathcal{F}$ (the function f_i and up to six additional functions that define the functions g_{ij} that form the frustum), in the sense that ϕ appears in the vertical decomposition of a 3D cell of $\mathcal{A}(D_\phi)$. By repeating this step for all cells of $\mathcal{M}_k^i$ and for all homothets K_i of $\mathcal{K}$, we obtain a decomposition of $\mathcal{M}_k = \mathcal{M}_k(\mathcal{K})$ into elementary cells. Finally, we lift each elementary cell of this decomposition vertically to $\mathcal{A}_k(\mathcal{F})$ in a straightforward manner, to obtain a corresponding decomposition of $\mathcal{A}_k(\mathcal{F})$. Kaplan et al. [28, Lemma 6.2] have shown that the complexity of the vertical decomposition of the $(\leq k)$-level $\mathcal{A}_{\leq k}(\mathcal{G})$ of a set of $\mathcal{G}$ of m bivariate semi-algebraic functions of constant complexity is $O(k^2\psi(n/k)\lambda_s(k))$, where $\psi(r)$ is the maximum complexity of the lower envelope of a subset of $\mathcal{G}$ of size at most r,

$\lambda_s(t)$ is the maximum length of Davenport-Schnizel sequences of order s composed of t symbols [36], and s is a constant depending on the complexity of the functions in $\mathcal{G}$. By applying the argument in [28] to our spherical coordinate system and using the worst-case bound $\psi(m) = O^*(m^2)$ on the complexity of the lower envelope of m constant-complexity bivariate functions [36], we conclude that the cells of $\mathcal{M}_k^i$ can be decomposed into $O^*(n^2k)$ elementary cells. Hence, $\mathcal{A}_k(\mathcal{F})$, or $\mathcal{M}_k(\mathcal{K})$, can be decomposed into $O^*(n^3k)$ elementary cells. By adapting the randomized incremental algorithm described in [28], $\mathcal{M}_k^i$, for each i, can be computed in $O^*(n^2k)$ expected time. We thus obtain:

Theorem 2.6. Let K be a compact, centrally-symmetric strictly convex semi-algebraic set in $\mathbb{R}^3$ of constant complexity. Let $\mathcal{K}$ be a set of n homothetic copies of K. The cells of $\mathcal{M}_k(\mathcal{K})$, and the cells of the k-level in the arrangement of their K-distance functions, can be decomposed into $O^*(n^3k)$ elementary cells, in $O^*(n^3k)$ expected time.

3 Vertical Shallow Cuttings

A key notion that we need for constructing the dynamic NN data structures of Section 4 is that of a vertical shallow cutting of $\mathcal{A}(\mathcal{F})$, as studied in [18, 21, 28] (for simpler scenarios). For a parameter k, a vertical k-shallow cutting of $\mathcal{F}$ is a collection of pairwise openly disjoint semi-unbounded pseudo-prisms (prisms for short), where each prism consists of all points that lie vertically below some pseudo-cone, of the decomposition (of Section 2) that covers $\mathcal{A}_{\leq k}(\mathcal{F})$, and is thus a semi-algebraic set of constant complexity. Furthermore, the ceilings of these prisms collectively form an x_4-monotone surface that lies between $\mathcal{A}_k(\mathcal{F})$ and $\mathcal{A}_{2k}(\mathcal{F})$, and each prism is crossed by the graphs of at most $O(k)$ functions of $\mathcal{F}$. The conflict list of a prism σ is the set of functions whose graphs cross σ. Following the ideas in [28], for a parameter $t \in [1, n]$, we construct a vertical (n/t)-shallow cutting as follows. We set two parameters: $r = \beta t \log t$, where β is a sufficiently large constant, independent of t, and $q = \beta \log t = r/t$. We choose a random subset $\mathsf{N} \subseteq \mathcal{F}$ consisting of r functions, construct $\mathcal{A}_q(\mathsf{N})$, and compute the decomposition of $\mathcal{A}_q(\mathsf{N})$ into a family Ξ of pseudo-cones as described in Section 2. For each pseudo-cone $\tau \in \Xi$, we associate a

label $\varphi(\tau)$ which is the function of N whose graph contains τ. Finally, we erect a semi-unbounded prism $\tau^{\uparrow}$ from each pseudo-cone τ of Ξ, given by

$$\tau^{\uparrow} = \bigcup \{\{c\} \times (-\infty, \rho] \mid (c, \rho) \in \tau\}.$$

Let $\Xi^{\uparrow}$ be the resulting set of pseudo-prisms. By Theorem 2.6, with large probability,

$$|\Xi| = O^*(r^3 q) = O^*(t^3 \log^4 t) = O^*(t^3).$$

We next show that Ξ is a vertical k-shallow cutting of $\mathcal{F}$, where $k = n/t$, with high probability.

Range spaces and shallow ε-nets. Let $\Sigma = (\mathsf{X}, \mathcal{R})$ be a (finite) range space, where X is a set of objects and $\mathcal{R} \subseteq 2^{\mathsf{X}}$ a set of ranges. Let $0 < \varepsilon < 1$ be a given parameter. A subset $\mathsf{N} \subseteq \mathsf{X}$ is called a shallow ε-net of Σ if it satisfies the following two properties for every range $R \in \mathcal{R}$ and for any parameter $\zeta \geq 0$: **(i)** If $|R \cap \mathsf{N}| \leq \zeta \log \frac{1}{\varepsilon}$ then $|R \cap \mathsf{X}| \leq \alpha(\zeta + 1)\varepsilon|\mathsf{X}|$, and **(ii)** If $|R \cap \mathsf{X}| \leq \zeta\varepsilon|\mathsf{X}|$ then $|R \cap \mathsf{N}| \leq \alpha(\zeta + 1) \log \frac{1}{\varepsilon}$. Here α is a suitable constant. Note the difference between shallow and standard ε-nets: Property (i) (with $\zeta = 0$) implies that a shallow ε-net is also a standard ε-net (possibly with a recalibration of ε). Property (ii) has no parallel in the case of standard ε-nets — there is no guarantee how a standard net interacts with small ranges (of the entire set X). The following result by Sharir and Shaul [37] is a generalization of the result on standard ε-nets [26].

Lemma 3.1 (Theorem 2.2 of Sharir and Shaul [37])**.** Let $\Sigma = (\mathsf{X}, \mathcal{R})$ be a range space with VC-dimension d. With a suitable choice of the constant of proportionality, a random sample $\mathsf{N} \subseteq \mathsf{X}$ of $O\left(\frac{d}{\varepsilon} \log \frac{1}{\varepsilon} + \log \frac{1}{\delta}\right)$ is a shallow ε-net with probability at least $1 - \delta$.

Ξ is a vertical shallow cutting. We apply the above result to the range space $\Sigma = (\mathcal{F}, \mathcal{R})$, so that each range in $\mathcal{R}$ is the subset of the surfaces of $\mathcal{F}$ that cross some region in $\mathbb{R}^4$, taken from some family Γ of regions. Concretely, this includes the following

families Γ: (i) the set of pseudo-cones generated by the decomposition of a set of at most seven functions of $\mathcal{F}$ (recall that a pseudo-cone is defined by at most seven functions), (ii) the set of pseudo-prisms erected on these pseudo-cones, (iii) the set of edges in an arrangement of five functions of $\mathcal{F}$ (an edge in the arrangement of a subset of $\mathcal{F}$ is defined by at most five functions), and (iv) the set of rays in the positive x_4-direction. Using standard arguments, it can be shown that the VC-dimension of Σ is finite. Therefore, by applying Lemma 3.1 with $\varepsilon = 1/t$ and $\delta = 1/3$ and choosing the constant of proportionality appropriately, the random subset $\mathcal{R}$ is a $(1/t)$-shallow net of Σ with probability at least $2/3$, so we assume that N is indeed a $(1/t)$-shallow net of Σ. Recall that $q = \beta \log t$. Assuming $\beta \geq 2\alpha$, where α is the constant in the definition of shallow nets, the converse of property (ii) of shallow nets, with $\zeta = 1$, implies that the level of any point p on $\mathcal{A}_q(\mathsf{N})$ with respect to $\mathcal{F}$ is at least n/t. Also, by property (i), the level of p is at most $O(n/t)$. Finally, the shallow net property also implies that the size of the conflict-list of any prism in $\Xi^{\uparrow}$ is $O(n/t)$. By testing each prism of $\Xi^{\uparrow}$ with every function of $\mathcal{F}$, the conflict list of all prisms can be computed in $O^*(nt^3)$ time. Hence, we obtain:

Theorem 3.2. Let K be a compact, centrally-symmetric strictly convex semi-algebraic set in $\mathbb{R}^3$ of constant complexity. Let $\mathcal{K}$ be a set of n homothetic copies of K. For any parameter $t \in [1, n-1]$, there exists a vertical (n/t)-shallow cutting Ξ of the arrangement of the distance functions of $\mathcal{K}$ of size $O^*(t^3)$. The cutting Ξ, along with the conflict list of each of its prisms, can be computed in $O^*(nt^3)$ expected time.

4 Dynamic Data Structures for Proximity Queries

In this section we describe a dynamic data structure for answering intersection-detection queries amid $\mathcal{K}$. Namely, for a query homothet $K_0 \in \mathbb{K}$, determine whether K_0 intersects some homothet of $\mathcal{K}$, and, if the answer is yes, return such a homothet. In addition to answering queries, the data structure can be updated efficiently as a homothet $K_0 \in \mathbb{K}$ is inserted into $\mathcal{K}$ or deleted from $\mathcal{K}$. The data structure can be extended to answering intersection-

reporting queries, namely, reporting all homothets of $\mathcal{K}$ that intersect K_0, at an additional cost of $O(k)$. Using the parametric-search technique (see, e.g., [10, 11]), a nearest-neighbor query under the K-distance can be reduced to answering $O(\log n)$ intersection-detection queries, so the cost increases by only a logarithmic factor. As already noted, the insertion of a new homothet into $\mathcal{K}$ can be handled using the standard dynamization technique by Bentley and Saxe [15] (see also [11]). That is, assume that we have an intersection-detection data structure that can handle deletions, which can be constructed in $P(n)$ time, so that a query can be answered in $Q(n)$ time and an object can be deleted in $D(n)$ time. Then the amortized insertion time is $O((P(n)/n)\log n)$, the deletion time remains $D(n)$, and the overall query time is $O(Q(n))$, assuming that $Q(n) = \Omega(n^{\varepsilon})$. See [15] for details. We first describe a data structure that answers a query in $O^*(1)$ time using $O^*(n^3)$ space and handles a deletion in $O^*(n^2)$ amortized expected time, and then describe a linear-size data structure that answers a query in $O^*(n^{2/3})$ time and handles a deletion in $O(\log^2 n)$ amortized expected time. Interpolating between these two setups, using standard techniques [2], for a storage parameter $s \in [n, n^3]$, we can construct a structure of size $O^*(s)$, in $O^*(s)$ expected time, that answers a query in $O^*(n/s^{1/3})$; the amortized expected update time is $O^*(s/n)$.

4.1 Fast query-time data structure

We follow the same approach of Agarwal and Matoušek for performing dynamic halfspace range reporting [11] (slightly improved but more involved approaches were proposed in [18, 28], but the one in [11] will do for our setting).

Data structure. Let $\mathcal{F}$ be the collection of distance functions corresponding to the elements of $\mathcal{K}$. Roughly speaking, we build a tree data structure on $\mathcal{F}$ using vertical shallow cuttings. That is, we start with $\mathcal{F}$, choose a parameter $t > 1$, compute an (n/t)-vertical shallow cutting Ξ of $\mathcal{F}$ of size $O^*(t^3)$, in $O^*(nt^3)$ time, create a child for each prism τ of Ξ, recursively construct the data structure for the conflict list $\mathcal{F}_\tau$ of every $\tau \in \Xi$, and attach it as the subtree of the child corresponding to τ. The recursion pro-

ceeds until we reach cells τ with $|\mathcal{F}_\tau| = O(1)$, in which case we just store $\mathcal{F}_\tau$ at τ, as a list, say. A query, with a copy $K_0 = (c_0, \rho_0)$ of K, is answered in a standard way, by traversing the tree in a top-down manner. At each node (prism) τ that we visit, we know that $(c_0, \rho_0) \in \tau$. We check whether (c_0, ρ_0) lies in any child prism of τ, and, if so, recurse at that child. Otherwise, we locate the child prism τ' for which (c_0, ρ_0) lies above τ', and report any element whose function belongs to the conflict list $\mathcal{F}_{\tau'}$ (or report all of them in the reporting version). However, this simple recursive approach runs into a technical difficulty when we delete a function from $\mathcal{F}$. When deleting a function from the root, the deletion has to be propagated to some of its children, to their children, and so on. On average, each function appears in the conflict lists of $O^*(t^2)$ prisms of Ξ. When such a "good" function is deleted, the deletion is propagated to only $O^*(t^2)$ descendants, leading to $O^*(n^2)$ overall deletion time, as can easily be verified. However, some of the functions (in fact, up to n/t of them) may appear in the conflict lists of $\Omega(t^3)$ prisms of Ξ. If these "bad" functions are the first n/t functions to be deleted and then the data structure is reconstructed, the overall cost of the deletion operations will be too high. As in [11, 28], we circumvent this difficulty by maintaining a partition of $\mathcal{F}$ into a few subsets and constructing a cutting for each subset that is good for all functions in that subset (i.e., each function in the subset appears in only $O^*(t^2)$ conflict lists). We now describe the data structure in detail. We fix a sufficiently large constant $t > 0$. The data structure is periodically reconstructed after performing some deletions. We use m to denote the size of $\mathcal{F}$ when the data structure was previously reconstructed and n to denote its current size. We reconstruct the data structure after deleting $m/2t$ functions from $\mathcal{F}$. Thus $n \geq m(1 - 1/2t)$. Let $P(r)$ denote the maximum time spent in constructing the data structure for a set of r functions. We pay for the reconstruction cost by charging $2tP(m - m/2t)/m \leq 2tP(n)/n$ time to each of the $m/2t$ delete operations that occurred before the reconstruction (the inequality holds because P is assumed to be superlinear). The data structure for $\mathcal{F}$ is a (recursively defined) tree $\Psi(\mathcal{F})$. We describe how a subtree $\Psi(\mathcal{G})$, for a subset $\mathcal{G}$ of ν functions, is constructed. If $\nu \leq n_0$, for some sufficiently large constant n_0, then $\Psi(\mathcal{G})$ is a single leaf, and we simply store $\mathcal{G}$ at that leaf. So assume that $\nu > n_0$. The root of $\Psi(\mathcal{G})$ stores the following data:

- A partition of $\mathcal{G}$ into subsets $\mathcal{G}_1, \ldots, \mathcal{G}_u$, $u \leq \lceil \log_2 t \rceil$, as described below.
- For each $1 \leq i \leq u$, a vertical (ν/t)-shallow cutting Ξ_i for $\mathcal{G}_i$. For each $\Delta \in \Xi_i$, we store its conflict list $\mathcal{G}_{i,\Delta}$.
- For each $1 \leq i \leq u$ and $\Delta \in \Xi_i$, a pointer to the tree $\Psi(\mathcal{G}_{i,\Delta})$.
- For each function $f \in \mathcal{G}_i$, the set $L_f \subseteq \Xi_i$ of prisms Δ crossed by the graph of f.
- A counter χ that keeps track of how many functions of $\mathcal{G}$ can be deleted before we reconstruct $\Psi(\mathcal{G})$.

After the construction of $\Psi(\mathcal{G})$, before any deletions occur, the following properties hold:

(P1) The counter χ is set to $\nu/2t$.

(P2) For each i, the x_4-monotone surface formed by the pseudocones of Ξ_i (i.e., the ceilings of prisms of Ξ_i) lie above $\mathcal{A}_{\nu/t}(\mathcal{G}_i)$.

(P3) For every i and $\Delta \in \Xi_i$, $|\mathcal{G}_{i,\Delta}| \leq cn/t$, for some suitable absolute constant c.

(P4) Each function $f \in \mathcal{G}_i$ appears in the conflict lists of at most $\kappa := 2C_1 t^{2+\delta}$ prisms of Ξ_i, where C_1 and $\delta > 0$ are the constants appearing in (1) below.

We now describe the construction of the partition $\mathcal{G}_1, \ldots, \mathcal{G}_u$, which proceeds iteratively. Suppose we have constructed $\mathcal{G}_1, \ldots, \mathcal{G}_{i-1}$. Set $\mathcal{G}_i^* = \mathcal{G} \setminus \bigcup_{j<i} \mathcal{G}_i$ and put $\nu_i = |\mathcal{G}_i^*|$. If $\nu_i \leq \nu/t$, we stop. Otherwise, set $t_i = t\nu_i/\nu$ and $k = \nu_i/t_i = \nu/t$. We construct a vertical k-shallow cutting Ξ_i of $\mathcal{G}_i^*$ of size $O(t_i^{3+\delta})$, for an arbitrarily small constant $\delta > 0$. For each $\Delta \in \Xi_i$, $|\mathcal{G}_{i,\Delta}^*| \leq c\nu_i/t_i = c\nu/t$. We note that

$$\sum_{\Delta \in \Xi_i} |\mathcal{G}_{i,\Delta}^*| \leq O(t_i^{3+\delta}) \frac{c\nu}{t} \leq C_1 \left(\frac{t\nu_i}{\nu}\right)^{3+\delta} \frac{\nu}{t} \leq C_1 \nu_i t^{2+\delta}. \tag{1}$$

A function $f \in \mathcal{G}_i^*$ is called good if it appears in the conflict list of at most κ prisms of Ξ_i and bad otherwise. Let $\mathcal{G}_i$ be the set of good functions of $\mathcal{G}_i^*$; set $\mathcal{G}_{i+1}^* = \mathcal{G}_i^* \setminus \mathcal{G}_i$. By (1), $|\mathcal{G}_{i+1}^*| \leq \nu_i/2$. Hence, $u \leq \lceil \log_2 t \rceil$. It is easily seen that properties (P1)–(P4) hold after the construction. This completes the description of the data structure. The total size and preprocessing time are $O^*(n^3)$.

Query procedure. Let $K(c_0, \rho_0) \in \mathbb{K}$ be a query homothet. Recall that $K(c_0, \rho_0)$ intersects a homothet of $\mathcal{K}$ if the point (c_0, ρ_0) lies above the lower envelope of the corresponding set $\mathcal{F}$ of functions. The query is performed as follows. We search $\Psi(\mathcal{F})$ in a top-down manner with (c_0, ρ_0). Suppose we are at the root of a subtree $\Psi(\mathcal{G})$. If $\Psi(\mathcal{G})$ is a leaf, we check all functions of $\mathcal{G}$ in a brute-force manner and return an answer in $O(n_0) = O(1)$ time. Otherwise, for each $i \leq u$, we check whether (c_0, ρ_0) lies above the lower envelope of $\mathcal{G}_i$. If the answer is yes for any i, we return yes and also return an intersecting homothet (see below). Otherwise, we return no. For a fixed $i \leq u$, we proceed as follows. We search with c_0, in a brute force manner, in $O^*(t^3) = O(1)$ time, to determine the pseudo-cone $\tau^{\downarrow}$ in the projection of Ξ_i that contains c_0. Let f_i be the function associated with τ, i.e., the ceiling of Δ lies in the graph of f_i. We test whether $\rho_0 \geq f_i(c_0)$. If so, K_i and $K(c_0, \rho_0)$ intersect, so we return yes and report K_i as an intersection witness and terminate the query. If not, (c_0, ρ_0) lies in the semi-unbounded prism Δ, and we continue the search recursively in the data structure $\Psi(\mathcal{G}_{i,\Delta})$. Let $Q(\nu)$ be the maximum time spent by the query procedure in the subtree $\Psi(\mathcal{G})$ storing at most ν functions. Then we obtain the following recurrence:

$$Q(\nu) \leq \lceil \log_2 t \rceil Q(c\nu/t) + O(t^{3+\delta}).$$

Its solution is easily seen to be $Q(\nu) = O(n^{\varepsilon})$, for any constant $\varepsilon > 0$, provided that n_0 and t are chosen sufficiently large.[3]

Deletion procedure. Let $f \in \mathcal{F}$ be a function that we wish to delete from $\mathcal{F}$. Again, we visit $\Psi(\mathcal{F})$ in a top-down manner. Suppose we are at the root v of a subtree $\Psi(\mathcal{G})$. If v is a leaf, then we simply delete f from $\mathcal{G}$ and stop. If v is an internal node, we first decrement the counter χ stored at v. If χ becomes 0, we reconstruct $\Psi(\mathcal{G})$ from scratch (for the current $\mathcal{G}$). Otherwise, we find the index i such that $f \in \mathcal{G}_i$. For each $\Delta \in L_f$, we delete f from $\mathcal{G}_{i,\Delta}$ and then recursively delete f from $\Psi(\mathcal{G}_{i,\Delta})$. By construction, $|L_f| \leq \kappa$. The deletion procedure maintains the properties (P3) and (P4), and (P2) is replaced with a slightly weaker property:

[3] As shown in [11], the query time can be improved to $O(\log n)$ by choosing $t = n^{\delta}$. The same improvement can be obtained here, but, for simplicity of exposition, we stay with the above weaker result.

(P2') For every i, the x_4-monotone surface formed by the pseudo-cones of Ξ_i (i.e., the ceilings of prisms of Ξ_i) lie above or on $\mathcal{A}_{\nu/2t}(\mathcal{G}_i)$.

The correctness of the query procedure follows from property (P2'). Following the analysis in [11], the amortized deletion time, including the time spent in reconstructing the data structure, is $O^*(n^2)$. Putting everything together, and combining this with the technique of Bentley and Saxe [15] for insertions, we obtain:

Lemma 4.1. $\mathcal{K}$ can be preprocessed, in $O^*(n^3)$ expected time, into a data structure of size $O^*(n^3)$, so that an intersection-detection query, including the cost of reporting an intersecting member of $\mathcal{K}$ if one exists, can be performed in $O^*(1)$ time. A homothet can be inserted into $\mathcal{K}$ or deleted from $\mathcal{K}$ in $O^*(n^2)$ amortized expected time.

4.2 Linear-size data structure

Next, we present a linear-size dynamic data structure that supports intersection-detection queries in $O^*(n^{2/3})$ time each, and can handle updates (insertions and deletions) in $O(\log^2 n)$ amortized expected time per update. For a homothet $K(c_0, \rho_0)$, let $f^+_{c_0,\rho_0} = \{(c, \rho) \in \mathbb{R} \mid \rho \geq f_{c_0,\rho_0}(c)\}$ be the region lying above the graph of the distance function f_{c_0,ρ_0}, which corresponds to the set of homothets that intersect $K(c_0, \rho_0)$. Put $\mathcal{K}^* = \{(c, \rho) \mid K(c, \rho) \in \mathcal{K}\} \subset \mathbb{R}^4$. For a query homothet $K(c_0, \rho_0)$, we wish to determine whether any point of $\mathcal{K}^*$ lies in $f^+_{c_0,\rho_0}$. We construct a linear-size partition tree for answering these queries, following the same approach as in [11, 34, 37]. We need a few definitions. Let $\mathbb{F}$ be the set of distance functions corresponding to the homothets in $\mathbb{K}$. Let $P \subset \mathbb{K}$ be a set of n points (representing homothets of K). For a parameter $k \geq 1$, we call a semi-algebraic set $\gamma \subset \mathbb{K}$, which is semi-unbounded in the $(-x_4)$-direction, k-shallow if $|P \cap \gamma| \leq k$. A major ingredient of the approach in [34, 37] is to construct a so-called test set composed of a small number of semi-algebraic sets, which represent well (in the sense made precise below) all distance functions of $\mathbb{F}$ that are (n/r)-shallow, for a given parameter $r > 1$, with respect to P. Formally, a finite collection Φ of semi-algebraic sets of constant complexity (not necessarily a subset of $\mathbb{F}$) is called a

test set for (n/r)-shallow ranges in $\mathbb{F}$ with respect to P if it satisfies the following properties:

(i) Every set in Φ is (n/r)-shallow with respect to P.

(ii) The complement of the union of any m sets of Φ can be decomposed into at most $\zeta(m)$ "elementary cells" (semi-algebraic sets of constant complexity) for any $m \geq 1$, where $\zeta(m)$ is a suitable monotone increasing superlinear function of m.

(iii) Any (n/r)-shallow set $\sigma \in \mathbb{F}$ can be covered by the union of at most δ ranges of Φ, where δ is a constant (independent of r).

Sharir and Shaul [37] showed that if there exists a test set of size $r^{O(1)}$ for (n/r)-shallow ranges of $\mathbb{F}$, then one can construct a linear-size partition tree on P so that an $\mathbb{F}$-emptiness query can be answered in $O^*(n/\zeta^{-1}(n))$ time, for the corresponding function ζ.

Test-set construction. We describe an algorithm for constructing a test set of size $O(r^4)$ with $\zeta(m) = O^*(m^3)$ and $\delta = 1$. Let $\mathcal{F}$ be the set of distance functions of $\mathcal{K}$ as above. We take a random subset $R \subseteq \mathcal{F}$ of $s = cr \log r$ functions, for some sufficiently large constant r, where $c > 0$ is another sufficiently large absolute constant, independent of, and much smaller than r, and construct the (standard) 4D vertical decomposition $\mathcal{A}^{\shortparallel}(R)$ of the arrangement $\mathcal{A}(R)$, of size $O^*(r^4)$, as described in [6, 32, 36]. By a standard random-sampling argument [26], with probability at least $1/2$, each cell of $\mathcal{A}^{\shortparallel}(R)$ is crossed by at most n/r functions of $\mathcal{F}$, provided that c is chosen sufficiently large. If this is not the case, we discard R and choose another random subset, until we find, in expected $O(1)$ trials, one with the desired property. We clip $\mathcal{A}^{\shortparallel}(R)$ within the halfspace $x_4 \geq 0$ (to restrict it to within $\mathbb{K}$). We choose a subset Ξ of the cells of $\mathcal{A}^{\shortparallel}(R)$, consisting of those cells that have at most n/r functions of $\mathcal{F}$ passing fully below them. By construction, these cells cover $\mathcal{A}_{\leq n/r}(\mathcal{F})$, and are contained in $\mathcal{A}_{\leq 2n/r}(\mathcal{F})$. Let τ be a cell of Ξ. Set $\Phi_\tau = \bigcup_{(c_0,\rho_0)\in\tau} f^+_{c_0,\rho_0}$. Since τ and f_{c_0,ρ_0} are semi-algebraic sets of constant complexity, Φ_τ is also a semi-algebraic set of constant complexity. For a homothet $K_i \in \mathcal{K}$, the

point $K_i^* \in \mathbb{K}$ lies in Φ_τ if and only if there is a point $(c_0, \rho_0) \in \tau$ such that $K_i^* \in f^+_{c_0,\rho_0}$. This happens when $(c_0, \rho_0) \in f_i^+$, i.e., the graph of the function f_i crosses τ or lies below τ. By construction, there are at most $n/r + n/r = 2n/r$ such functions. Consequently, Φ_τ is $(2n/r)$-shallow with respect to the points of $\mathcal{K}^*$ with $\delta = 1$. Set $\Phi = \{\Phi_\tau \mid \tau \in \Xi\}$. Φ is a family of $O^*(r^4)$ constant-complexity semi-algebraic sets in $\mathbb{K}$, each of which is $(2n/r)$-shallow with respect to $\mathcal{K}^*$. This is the desired test set. Each set Φ_τ is unbounded in the positive x_4-direction, therefore the complement of the union of a subset of m sets is the region lying below their lower envelope. By a result in [5], the complement of their union (i.e., the region below their lower envelope) can be decomposed into $O^*(m^3)$ pseudo-prisms. Hence, we obtain:

Lemma 4.2. Let $P \subset \mathbb{K}$ be a set of n points and $r \geq 1$ a parameter. A test set Φ of size $O(r^4)$ for the functions in $\mathbb{F}$ that are (n/r)-shallow with respect to P can be computed in $O^*(r^4)$ expected time, so that $\zeta(m) = O^*(m^3)$ and $\delta = 1$.

Using Lemma 4.2 and adapting the approach in [37], we can preprocess $\mathcal{K}$ into a linear-size data structure that supports intersection-detection queries in $O^*(n^{2/3})$ time. By (re)constructing portions of the partition tree periodically (as in the previous data structure), deletions can be handled efficiently. Omitting the straightforward details, we obtain:

Lemma 4.3. Let $\mathcal{K}$ be a set of n homothets of K. $\mathcal{K}$ can be preprocessed, in $O(n \log n)$ expected time, into a data structure of size $O(n)$, so that an intersection-detection query can be answered in $O^*(n^{2/3})$ time. A homothet of K can be inserted into or deleted from the data structure in $O(\log^2 n)$ amortized time.

By combining this data structure with Lemma 4.1 in a standard manner [2], we can obtain the following space/query-time trade-off:

Theorem 4.4. Let $\mathcal{K}$ be a set of n homothets of K, and let $s \in [n, n^3]$ be a storage parameter. $\mathcal{K}$ can be preprocessed, in $O^*(s)$ expected time, into a data structure of size $O^*(s)$, so that an intersection-detection query can be answered in $O^*(n/s^{1/3})$ time. A homothet can be inserted into or deleted from the data structure in $O^*(s/n)$ amortized time.

Remark 1. (i) Using standard techniques [11,34], the above data structure can be adapted in a straightforward manner to report all k homothets intersected by a query homothet in an additional $O(k)$ time. (ii) The above theorem, with $s = n^{3/2}$, implies that a sequence of n insertion/deletion/query operations on the data structure can be performed in $O^*(n^{3/2})$ expected time.

Finally, by combining this data structure with the parametric-search technique [10], we establish Theorem 1.1 (for answering nearest-neighbor queries in $\mathcal{K}$ under the K-distance).

5 Breadth-First Search, Reverse Shortest Paths, and Related Problems

In this section we present applications of our techniques to various basic graph algorithms on intersection or proximity graphs of a set of homothets in $\mathbb{R}^3$. In this section we present applications of our techniques to the reverse shortest path problem.

Breadth-first search. We follow the standard high-level approach. We run the BFS layer by layer, where the first layer contains just the start object $K(c_s, \rho_s)$. Consider the step where we have constructed layer L_i and wish to construct the next layer L_{i+1}. Let $\mathcal{U}_i$ denote the set of all members of $\mathcal{K}$ that the BFS has not yet reached (up to this step). We maintain $\mathcal{U}_i$ in the dynamic data structure of Section 4, which supports deletions and queries (insertions are irrelevant here), where a query specifies an object $K(c_0, \rho_0)$ (in L_i), and wants to determine whether $K(c_0, \rho_0)$ intersects some object $K(c, \rho)$ of $\mathcal{U}_i$, and, if so, to return one such object (or, alternatively, report all intersecting homothets in $\mathcal{U}_i$). The output object, or objects, if they exist, are added to L_{i+1}, and are immediately removed from the structure (and from $\mathcal{U}_i$), to make sure that they are not reported again (by other elements of L_i, or by elements of future layers). The BFS procedure executes a total of $O(n)$ queries and deletions, therefore by Remark 1 (following Theorem 4.4, with $s = n^{3/2}$), the overall cost of the the algorithm is $O^*(n^{3/2})$, as stated in Theorem 1.3 (a).

Depth-first search. The same machinery can be used to perform depth-first search on the graph $\mathcal{G}(\mathcal{K})$. At each step we are at some homothet $K(c_0, \rho_0) \in \mathcal{K}$, and want to find an intersecting neighbor that has not yet been visited. As above, we maintain the set $\mathcal{U}$ of all members of $\mathcal{K}$ that the DFS has not yet reached, using the dynamic data structure of Section 4, with the same choice of $s = n^{3/2}$. Using this structure, we find a neighbor of $K(c_0, \rho_0)$ (if one exists), delete it from $\mathcal{U}$, and continue the search at that neighbor (or back up to the parent node, or start a new DFS tree). As above, since we perform $O(n)$ searches and deletions, the overall cost is $O^*(n^{3/2})$ time. This, together with the preceding analysis, establish Theorem 1.3 (a).

Reverse shortest paths. We next use Theorem 1.3 to solve the corresponding reverse shortest path (RSP) problem, defined as follows. Let $\mathcal{K}$ be as above, and let $r > 0$ be a parameter. We consider the setup where, for each $K(c, \rho) \in \mathcal{K}$, we expand its size by adding r to the size of each element of $\mathcal{K}$, i.e., replacing each $K(c, \rho)$ by $K(c, \rho + r)$. Define $\mathcal{G}_r(\mathcal{K})$ to be the intersection graph of the expanded homothets in $\mathcal{K}$. We are then given two elements $K(c_s, \rho_s)$, $K(c_t, \rho_t)$ of $\mathcal{K}$ and an integer $1 \leq k < n$, and seek the smallest value r^* for which $\mathcal{G}_{r^*}(\mathcal{K})$ contains a path from $K(c_s, \rho_s + r^*)$ to $K(c_t, \rho_t + r^*)$, of at most k edges. This is the reverse shortest path (RSP) problem for that graph. The decision problem associated with this RSP problem is, given r, to decide whether $\mathcal{G}_r(\mathcal{K})$ contains such a path. If the answer is positive (resp., negative) then $r^* \leq r$ (resp., $r^* > r$). Solving this decision problem is trivially performed using BFS on $\mathcal{G}_r(\mathcal{K})$, starting from $K(c_s, \rho_s + r)$, so, by Theorem 1.3, the resulting decision procedure runs in $O^*(n^{3/2})$ time. We now solve the RSP problem using the shrink-and-bifurcate technique (see [14,20,27]), adapted to the setup at hand. The critical values of r are values at which some comparison involving r that the BFS algorithm performs changes its outcome. These critical values are of several kinds. First, these are values of r at which two expanded homothets, $K(c, \rho + r)$, $K(c', \rho' + r)$, become externally tangent to one another (they are disjoint for smaller values of r

and intersect from this r and above). This happens when

$$\text{dist}_K(c, c') = (\rho+r)+(\rho'+r) = \rho+\rho'+2r, \text{ or } r = \frac{1}{2}\left(\text{dist}_K(c, c') - \rho - \rho'\right). \tag{2}$$

Other kinds of critical values arise in comparisons that occur during the construction of, and search in, the data structure of Section 4. There are many kinds of such comparisons, each of which either involves the manipulation of $O(1)$ functions $f_{c,\rho}$ (that is, $f_{c,\rho+r}$), or comparisons between points and functions, which determine whether the point lies above, on, or below the function. The latter kind of comparisons gives rise to critical values of the first kind (given in (2)). The shrinking part of the technique receives a parameter L, and aims to construct an interval I that contains r^* and at most L other critical values. This procedure is based on a variant of "distance selection", which in turn is based on a procedure for counting the number of critical values that are smaller than or equal to r_0, for some query parameter r_0. This procedure is implemented using off-line semi-algebraic range searching. Specifically, for critical values in (2), each homothet $K(c, \rho)$ is mapped to the point $(c, \rho) \in \mathbb{R}^4$, as before, and also to the range

$$\sigma(c, \rho) = \{(c', \rho') \in \mathbb{R}^4 \mid f_{c,\rho}(c') = \text{dist}_K(c, c') - \rho \leq \rho' + 2r_0\}.$$

The goal is then to count the number of point-range containments. Each range is a semi-algebraic region of constant complexity, and both points and ranges have four degrees of freedom. The general technique for offline semi-algebraic range searching, as presented, e.g., in [2, Appendix], runs in this case in time $O^*(n^{8/5})$, which implies that, using the improved procedure of Chan and Huang [20], interval shrinking (restricted to this type of critical values) can be performed in $O^*(n^{8/5}/L^{4/5})$ expected time. To handle critical values of other kinds, we observe that our additive expansion rule subtracts the same value r from each of these functions, so comparisons involving any set of $O(1)$ functions can be determined independently of r. Put differently, we can think of the process as constructing the data structure without r (i.e., with $r = 0$), in a manner that requires no simulation, and then query the structure with $K(c_0, \rho_0 + 2r)$. Hence the only r-dependent comparisons occur during a query. Unfortunately, here there is a

significant difference between the fast-query version of the structure and the linear-space version. In the fast-query version, the query is with the point $(c_0, \rho_0 + 2r)$, and each r-dependent comparison tests the position of this point (above, on, below) with respect to a function in $\mathcal{F}$. In other words, all the corresponding critical values are of the kind in (2). In contrast, in the linear-storage version, the query is with a surface, dual to the point $(c_0, \rho_0 + 2r)$, and certain steps in the execution of a query seek cells of the current decomposition that are intersected by the surface or lie fully below it, and these operations, which do depend on r, generate critical values that are not of the type (2). In principle, this is not a real problem. As just briefly reviewed, the shrinking stage (see [14, 20, 27]), relies on a "distance-selection" procedure, which is implemented as an off-line semi-algebraic range-counting procedure, where the ranges are defined by some semi-algebraic predicate of constant complexity. For critical values of the former kind, the predicate is given by (2). For critical values of the other kinds, we just have to use different predicates. Here too the queries have four degrees of freedom (defined by the parameters $(c_0, \rho_0 + 2r)$), but the data objects (e.g., a vertex of a cell in the decomposition) may have many more degrees of freedom, and this will slow down the procedure. There are two ways out of this difficulty. One approach would be to bound the maximum number of degrees of freedom of the data objects, which is some small constant, and then run the relevant range searching procedure, using, e.g., the analysis in [2, Appendix], with the inferior time bound that it would yield. The other approach, which is the one that we follow, is to give up in the simulation the use of the linear-space version of the structure, and just use the fast-query version. Recall that this version, applied to a set of n homothetic copies, uses a total of $O^*(n^3)$ storage and preprocessing, answers a query in $O^*(1)$ time, and performs a deletion in $O^*(n^2)$ (amortized) time. To make it more efficient, we partition $\mathcal{K}$ into $O(n/t)$ subsets, each consisting of at most t homothets, for a suitable parameter t that will be fixed shortly, and construct a different structure for each subset. A query has now to be performed on each of these structures, for a total of $O^*(n/t)$ time, while a deletion has to be performed only within its own subset, for a cost of $O^*(t^2)$ amortized expected time. Choosing $t = n^{1/3}$ makes the two bounds asymptotically the same, for a total cost (for $O(n)$

queries and deletions) of $O^*(n^{5/3})$. The standard analysis of the bifurcation stage [14] yields the bound $O^*(L^{1/2}D(n))$ on its running time, where $D(n)$ is the cost of the decision procedure. Here however, we need to modify the analysis because we have two versions of the decision procedure, the one that we simulate, and the one that runs, without simulation, on some concrete values r_0 of r. The respective running time bounds of these versions are $D_1(n) = O^*(n^{5/3})$ and $D_0(n) = O^*(n^{3/2})$. Here is a sketch of the modified analysis (see [14, 27] for full details). The standard analysis uses two parameters X, Y, where X is a threshold on the number of unresolved comparisons accumulated in a single phase of the bifurcation, and Y is a threshold on the total amount of simulation steps performed in a phase. The cost of a phase is $O^*(XY + D_0(n))$, where the second term results from the binary search for r^* at the end of the phase. We take $XY = D_0(n)$. The number of phases is $O\left(\frac{L}{X} + \frac{D_1(n)}{Y}\right)$, which we optimize by making these two terms equal, i.e., by choosing $Y = \frac{D_1(n)X}{L}$. Solving this system of equations, we get $X = L^{1/2}\left(\frac{D_0(n)}{D_1(n)}\right)^{1/2}$, and the overall cost is then

$$O^*\left(XY \cdot \frac{D_1(n)}{Y}\right) = O^*(D_1(n)X) = O^*(L^{1/2}D_0(n)^{1/2}D_1(n)^{1/2}).$$

In our case, $D_1(n) = O^*(n^{5//3})$ and $D_0(n) = O^*(n^{3/2})$, so the overall bound is

$$O^*\left(\frac{n^{8/5}}{L^{4/5}} + L^{1/2}n^{3/4}n^{5/6}\right) = O^*\left(\frac{n^{8/5}}{L^{4/5}} + L^{1/2}n^{19/12}\right).$$

To balance the two terms, we choose $L^{13/10} = n^{1/60}$, or $L = n^{1/78}$, and the bound then becomes $O^*\left(n^{62/39}\right)$. That is, we have obtained the bound asserted in Theorem 1.4.

Remark 2. When K is the Euclidean ball, the algorithm becomes simpler, and its performance can be improved to $O^*(n^{56/39})$, as recently shown in [30]. The reason is that the intersection detection subprocedure can be transformed to a dynamic halfspace range reporting in $\mathbb{R}^5$, using a simple lifting transform. The latter task can be solved using the machinery of Agarwal and Matoušek [11], which leads to the above improved bound. See [30] for full details.

Minimum spanning forests and Dijkstra's shortest path algorithm. Let P be a set of n points in $\mathbb{R}^3$, and let K be as above. As we recall, for a given $r_0 > 0$, the r_0-proximity graph $\mathcal{G}_{r_0}(P)$ (with respect to the metric $\|\cdot\|_K$) is the graph whose vertices are the points of P, and whose edges are all the pairs (p, q) such that $\text{dist}_K(p, q) \leq r_0$. We assign to each edge $(p, q) \in G_{r_0}(P)$ the weight $\text{dist}_K(p, q)$. We consider the problems of computing a minimum spanning forest of $\mathcal{G}_{r_0}(P)$ (note that $\mathcal{G}_{r_0}(P)$ might be disconnected), and of computing the shortest-path tree in $\mathcal{G}_{r_0}(P)$ from some start point K_s. For the minimum spanning forest problem, we run Prim's algorithm which, at each step, maintains the partition of P into the subsets V, U, of visited and unvisited points, respectively. The algorithm computes the bichromatic closest pair (BCP) (p, q) in $V \times U$. If $\text{dist}_K(p, q) \leq r_0$, we add the edge (p, q) to the MST and move q from U to V. Otherwise, we conclude that the vertices of U form a connected component of $\mathcal{G}_{r_0}(P)$, we discard U, and continue running Prim's algorithm on V (this case will not arise when $\mathcal{G}_{r_0}(P)$ is connected). By Corollary 1.2, the BCP of U and V can be maintained in $O^*(n^{1/2})$ amortized expected time under insertion/deletion of points, so the overall expected run time of the algorithm is $O^*(n^{3/2})$, as claimed in Theorem 1.3 (b). The implementation of Dijkstra's shortest-path algorithm is similar. At each step of the algorithm, we have the sets V and U of nodes that the algorithm has already visited, and those that it did not, respectively. Each point $p \in V$ has a weight $\omega(p)$ associated with it, equal to the shortest distance from the start point s to p. We now compute the BCP in $V \times U$, where the points $p \in V$ are additively weighted by the corresponding distances $\omega(p)$, and the points of U are unweighted. At each step, we wish to compute the weighted BCP of $V \times U$, namely $(p^*, q^*) = \arg\min_{(p,q)\in V\times U} (\text{dist}_K(p, q) + \omega(p))$. If $\text{dist}_K(p^*, q^*) \leq r_0$, we add q^* as a child of p^*, and move q^* from U to V, with weight $\omega(q^*) = \omega(p^*) + \text{dist}_K(p^*, q^*)$. Otherwise, we conclude that all remaining points of U are farther than r_0 from p^*, so none of them is adjacent to p^*. We delete p^* from V and continue. We implement the computation of (p^*, q^*) using Corollary 1.2, as follows. We choose a sufficiently large parameter A, larger than $\max_p \omega(p)$, which can easily be computed in advance in $O(n)$ time. Then we replace each point $p \in V$ by the homothetic copy $K_p = p + (A - \omega(p))K$. Set $\mathcal{K}_V = \{K_p \mid p \in V\}$. It is easily seen that if (K_{p^*}, q^*) is the BCP of $\mathcal{K}_V \times U$, then (p^*, q^*) is the weighted

BCP of V and U. Hence, each step of the algorithm can be implemented in $O^*(n^{1/2})$ amortized expected time. At each step of the algorithm, we either move a point from U to V or delete a point from V. Since each point of P is moved from U or deleted from V at most once, the overall expected time is $O^*(n^{3/2})$, as claimed in Theorem 1.3 (b).

References

[1] P. K. Agarwal, Simplex range searching and its variants: A review, in Journey through Discrete Mathematics: A Tribute to Jiří Matoušek, M. Loebl, J. Nešetřil and R. Thomas, editors, Springer Verlag, Berlin-Heidelberg, 2017, pages 1–30.

[2] P. K. Agarwal, B. Aronov, E. Ezra, M. J. Katz, and M. Sharir, Intersection queries for flat semi-algebraic objects in three dimensions and related problems, ACM Trans. Algorithms 21 (2025), 25:1–25:59.

[3] P. K. Agarwal, B. Aronov, E. Ezra, and J. Zahl, An efficient algorithm for generalized polynomial partitioning and its applications, SIAM J. Comput. 50 (2021), 760–787.

[4] P. K. Agarwal, B. Aronov, M. Sharir and S. Suri, Selecting distances in the plane, Algorithmica 9 (1993), 495-–514.

[5] P. K. Agarwal, E. Ezra and M. Sharir, Vertical decomposition in 3D and 4D with applications to line nearest-neighbor searching in 3D, in preparation. A preliminary version in Proc. 35th ACM-SIAM Sympos. Discrete Algorithms, 2024, 150–170. Also in arXiv:2311.01597.

[6] P. K. Agarwal, A. Efrat and M. Sharir, Vertical decomposition of shallow levels in 3-dimensional arrangements and its applications, SIAM J. Comput. 29(3) (1999), 912–953.

[7] P. K. Agarwal and J. Erickson, Geometric range searching and its relatives, In Advances in Discrete and Computational Geometry, volume 223 of Contemp. Math., pages 1–56. AMS Press, Providence, RI, 1999.

[8] P. K. Agarwal, H. Kaplan, M. J. Katz and M. Sharir, Segment proximity graphs and nearest neighbor queries amid disjoint segments, Proc. 32nd Annu. European Sympos. Algorithms, 2024, 7:1–7:20.

[9] P. K. Agarwal, M. J. Katz and M Sharir, On reverse shortest paths in geometric proximity graphs, Comput. Geom. 117 (2024), Art. 102053.

[10] P. K. Agarwal and J. Matoušek, Ray shooting and parametric search, SIAM J. Comput. 22 (1993), 794–806.

[11] P. K. Agarwal and J. Matoušek, Dynamic halfspace range reporting and its applications, Algorithmica 13 (1995), 325–345.

[12] A. Andoni and P. Indyk, Nearest neighbors in high dimensional spaces, in Handbook of Discrete and Computational Geometry (eds. J.E. Goodman, J. O'Rourke, and C. D. Tóth), 3rd edition, CRC Press, Boca Raton, FL, 2017.

[13] S. Basu, R. Pollack, and M.-F. Roy, Algorithms in Real Algebraic Geometry, 2nd Edition, Springer Verlag, Berlin, 2006.

[14] R. Ben Avraham, O. Filtser, H. Kaplan, M. J. Katz and M. Sharir, The discrete and semicontinuous Fréchet distance with shortcuts via approximate distance counting and selection, ACM Trans. Algorithms 11(4) (2015), 29:1–29:29.

[15] J. L. Bentley and J. B. Saxe, Decomposable searching problems I: Static-to-dynamic transformation, J. Algorithms 1 (1980), 301–358.

[16] M. de Berg and S. Cabello, An $O(n \log n)$ algorithm for single-source shortest paths in disk graphs, Proc. 33rd Annual European Sympos. on Algorithms, 2025, 81:1–81:15.

[17] S. Cabello and M. Jejčič, Shortest paths in intersection graphs of unit disks, Comput. Geom. Theory Appls. 48 (2015), 360–367.

[18] T. M. Chan, A dynamic data structure for 3D convex hulls and 2D nearest-neighbor queries, J. ACM 57 (2010), 16:1–16:15.

[19] T. M. Chan, Dynamic generalized closest pair: Revisiting Eppstein's technique, Proc. 3rd Sympos. Simplicity in Algorithms, 2020, 33–37.

[20] T. M. Chan and Z. Huang, Faster algorithms for reverse shortest path in unit-disk graphs and related geometric optimization problems: Improving the shrink-and-bifurcate technique, Proc. 41st Intl. Sympos. Comp. Geom., 2025, 32:1–32:14.

[21] T. M. Chan and K. Tsakalidis, Optimal deterministic algorithms for 2-d and 3-d shallow cuttings, Discrete Comput. Geom. 56 (2016), 866–881.

[22] B. Chazelle, H. Edelsbrunner, L. Guibas and M. Sharir, A singly exponential stratification scheme for real semi–algebraic varieties and its applications, Proc. 16th Int. Colloq. on Automata, Languages and Programming (1989) pp. 179–193. Also in Theoretical Computer Science 84 (1991), 77–105.

[23] K. L. Clarkson, Nearest neighbor searching and metric space dimensions, in Nearest-Neighbor Methods in Learning and Vision (eds. G. Shakhnarovich, T. Darrell, and P. Indyk), The MIT Press, 2006.

[24] K. L. Clarkson and P. W. Shor, Applications of random sampling in computational geometry II, Discrete Comput. Geom. 4 (1989), 387–421.

[25] D. Eppstein, Fast hierarchical clustering and other applications of dynamic closest pairs, Discrete Comput. Geom. 13 (1995), 111–122.

[26] D. Haussler and E. Welzl, Epsilon nets and simplex range queries, Discrete Comput. Geom. 2 (1987), 127–151.

[27] H. Kaplan, M. J. Katz, R. Saban and M. Sharir, The unweighted and weighted reverse shortest path problem for disk graphs, ACM Trans. Algorithms, to appear. Also in Proc. 31st Annu. European Sympos. Algorithms, 2023, 67:1–67:14.

[28] H. Kaplan, W. Mulzer, L. Roditty, P. Seiferth and M. Sharir, Dynamic planar Voronoi diagrams for general distance functions and their algorithmic applications, Discrete Comput. Geom. 64 (2020), 838–904.

[29] M. J. Katz, R. Saban and M. Sharir, Near-linear algorithms for visibility graphs over a 1.5-dimensional terrain, Proc. 3nd Annu. European Sympos. Algorithms, 2024, 77:1–77:17.

[30] M. J. Katz, R. Saban and M. Sharir, BFS and reverse shortest paths for ball intersection graphs in three and higher dimensions, Proc. 36th Internat. Sympos. Algorithms and Computation (2025), 45:1–45:15.

[31] K. Klost, An algorithmic framework for the single source shortest path problem with applications to disk graphs, Comput. Geom., 111 (2023), Art. 101979.

[32] V. Koltun, Almost tight upper bounds for vertical decompositions in four dimensions, J. ACM 51 (2004), 699–730.

[33] C.-H. Liu, Nearly optimal planar k nearest neighbors queries under general distance functions, SIAM J. Comput. 51(3) (2022), 723–765.

[34] J. Matoušek, Reporting points in halfspaces, Comput. Geom. Theory Appl. 2 (1992), 169–186.

[35] J. Matoušek and Z. Patáková, Multilevel polynomial partitioning and simplified range searching, Discrete Comput. Geom. 54 (2015), 22–41.

[36] M. Sharir and P. K. Agarwal, Davenport-Schinzel Sequences and Their Geometric Applications, Cambridge University Press, New York, 1995.

[37] M. Sharir and H. Shaul, Semialgebraic range reporting and emptiness searching with applications, SIAM J. Comput. 40(4) (2011), 1045–1074.

[38] H. Wang and Y. Zhao, Reverse shortest path problem for unit-disk graphs, Proc. 17th Algorithms and Data Structures Sympos., 2021, 655–668.